

Vibe Coding / Agentic Flow 面试题集

注: 本题集不存在唯一标准答案, 所附“参考答案”仅为抛砖引玉. 适用对象: 每天使用 Claude Code、Codex 等 agentic CLI 工具的开发者. 参考资料:
<https://code.claude.com/docs/llms.txt>、
<https://github.com/openai/codex/blob/main/docs/getting-started.md> 以及
Anthropic / OpenAI 公开文档.

目录

- 一、背景知识篇
 - 二、细节梳理篇
 - 三、工作流篇
 - 四、系统设计篇 (开放思辨)
 - 五、概念哲学篇 (开放思辨)
-

一、背景知识篇

Q1. 什么是 `/command`? 和 `skill` 的区别是什么? 什么时候用 `command` (已 deprecated), 什么时候用 `skill`?

参考答案:

- **`/command` (Slash Command):** 早期 Claude Code 的扩展机制. 用户在 `.claude/commands/` 下放置 `.md` 文件, 通过在对话中输入 `/command-name` 来触发. 本质是“用户主动调用的 prompt 片段”. 当前在 Claude Code 中已被官方标记为 deprecated, 由 `skill` 替代.
- **Skill:** 新一代扩展机制, 位于 `.claude/skills/<name>/SKILL.md` (或 `~/.claude/skills/`、`plugin` 中). 区别在于:
 1. 可被模型自主调用 —— Skill 在 YAML frontmatter 里写 description, Claude 会根据语义判断是否启用, 而不仅靠用户敲 `/`.

2. 支持附属文件 —— skill 目录里可放脚本、参考文档、子模板.
 3. 生命周期管理 —— skill 在 /compact 时会被重新注入 (前 5K token, 全局 25K 预算).
 4. 细粒度受控 —— frontmatter 可指定 allowed-tools、disable-model-invocation、paths (路径范围限定)、effort、model.
- 何时使用:
 - 优先 **skill**: 几乎所有新场景, 尤其是希望模型在合适时机自动启用、或需要附带脚本/示例文件的.
 - **command** 仍合理的少数场景: 完全确定性的、必须人工显式触发的一行 prompt 包装 (例如某些团队遗留资产); 但新项目不再推荐.

Q2. 什么是 agent? 什么情况下我们使用 agent, 什么情况下使用 skill?

参考答案:

- **Agent (Sub-agent)**: 一个拥有独立 system prompt、独立上下文窗口、独立工具白名单的“小型 Claude”. 在 Claude Code 中通过 `.claude/agents/<name>.md` 定义, 由主对话通过 Agent 工具派发. Sub-agent 完成任务后只把“总结”回传给主对话, 中间过程不污染主上下文.
- **Skill**: 一段供模型读取的指令/规程 (instructions), 它不开新上下文, 而是在当前对话里指导 Claude 接下来怎么做.

区别本质:

维度	Skill	Agent
上下文	共享主上下文	独立上下文
调用代价	低 (只是注入文本)	高 (一次完整推理回合)
适合	“怎么做”的指导、规范、check-list	大量工具调用、可能产生大量噪声的子任务
结果	改变后续主对话行为	返回一段总结

- 决策原则:

- 任务会产生大量工具输出(读几十个文件、跑测试、抓日志) → 用 **agent**, 避免污染主上下文.
 - 任务是告诉 **Claude** 一种规程(比如“提交前先跑 lint”) → 用 **skill**.
 - 任务是只读的探索/搜索 → 优先用 **Explore** 这类内置 agent.
 - 任务需要多步骤设计且要保留思考过程在主对话 → 不开 agent, 让主对话自己做.
-

Q3. Sub-agent 是什么? Sub-agent 之间可以互相通信吗?

参考答案:

- **Sub-agent:** 见 Q2. 关键特征是“独立上下文 + 独立工具集 + 只回传 summary”.
 - 能否互通?
 - 默认不能直接通信. Claude Code 的 sub-agent 通信模型是星型: 所有 sub-agent 只与主对话通信, 主对话作为 hub 在它们之间转发信息.
 - 实验性 **Agent Teams** (需要开启 `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1`) 引入了 `SendMessage` 工具, 才允许同级 agent 间用 ID 直接发消息; 但这是另一套模型 (见 Q4).
 - **设计原因:** 强制信息汇聚到主对话, 保证状态可追溯、可审计, 也防止 agent 之间形成不可控的循环对话.
-

Q4. Agent Team 和 Sub-agent 之间最大的区别是什么?

参考答案:

- **Sub-agent:** “雇一个临时工去做一件事, 回来汇报”. 一次性、星型、汇报式. 主对话发起 → sub-agent 执行 → 返回 summary → sub-agent 即销毁.
- **Agent Team:** “组建一个长期协作的团队, 成员之间可以互相喊话”. 持续性、可互通、并行.
 - 每个 teammate 拥有持久的独立上下文.
 - 通过 `SendMessage(to=<agent_id>, message=...)` 直接互发消息.
 - 适合“多个长期角色并行推进”的场景 (例如 frontend 工程师 + backend 工程师 + reviewer 同时工作).

- 最大区别:
 - 拓扑: Sub-agent 是”主→子”的树; Agent Team 是”成员之间互通”的图.
 - 生命周期: Sub-agent 是 fire-and-forget; Teammate 是长期存在.
 - 状态共享: Sub-agent 只返回最终 summary; Teammate 之间可来回讨论, 维护持续上下文.
- 取舍: Agent Team 更接近”多 agent 协作”研究方向, 但也更难调试、更容易上下文爆炸; 大多数日常任务用 sub-agent 就够.

Q5. MCP 是什么? 它和 API 接口的区别是什么?

参考答案:

- **MCP (Model Context Protocol):** Anthropic 主导、现已开源的协议, 用于把”外部工具/资源/prompt”以**标准格式**暴露给大模型. 一个 MCP server 进程可同时声明它提供哪些 tools、resources、prompts, 客户端 (Claude Code、Claude Desktop、Codex 等) 通过统一协议发现并调用. **与”直接调 API”的区别:**

维度	裸 API	MCP
描述方式	自然语言 + 文档, 每个 LLM/客户端自己实现适配	标准 schema, 一次声明全网客户端通用
工具发现	需要预先告知模型	客户端连接时自动 list tools
鉴权	各 API 自己一套	标准 OAuth 2.0, 支持 dynamic client registration
传输	HTTP 各异	stdio / HTTP / SSE 三种标准传输
复用	每个 LLM 平台都要重做	一份 MCP server, 所有支持 MCP 的客户端可用

- **类比:** API 是”USB 线”, 每家形状不一样; MCP 是”USB-C”, 给所有 LLM 客户端统一接口.
 - **本质区别:** MCP 不仅传数据, 还自**描述能力** —— 工具签名、参数 schema、是否可写、是否需要确认, 模型据此推理”何时调用、怎么调用”.
-

Q6. Sub-agent 可否再衍生 (派生) 它自己的 sub-agents?

参考答案:

- 不可以. Claude Code 官方文档明确说明: sub-agent 无法再启动 sub-agent.
 - 原因:
 1. 防止无限递归 —— 一旦允许嵌套, agent 树深度不可控, 调试和成本控制变成噩梦.
 2. 保持信息汇聚 —— 所有总结必须回到主对话, 才能让用户和主 agent 有完整可审计的轨迹.
 3. 避免上下文窗口指数爆炸 —— 嵌套调用每层都是独立 context, 容易把账单和延迟拉爆.
 - 如果确实需要“嵌套委托”, 怎么办?
 - 在 sub-agent 里使用 **skill** 来组织内部步骤 (skill 不开新上下文).
 - 让主对话串行或并行地多次派发 sub-agent (扁平化).
 - 使用 Agent Team 模型 (但那是平级互通, 不是嵌套).
-

二、细节梳理篇

Q7. CLAUDE.md / AGENTS.md 是什么? 加载顺序和优先级如何?

参考答案:

- **CLAUDE.md (Claude Code) / AGENTS.md (Codex):** 是项目/用户级别的“持久 prompt 注入”, 在每次 session 启动时被自动加载到 system prompt, 用于声明“我希望 agent 一直记住的规则”.
- **加载层级 (Claude Code):**
 1. **Managed policy** —— 企业管控级, 由 IT 部署, 不可被覆盖.
 2. **User** —— ~/.claude/CLAUDE.md, 个人全局规则.
 3. **Project** —— 仓库根 ./CLAUDE.md, 项目级规则.
 4. **Local** —— CLAUDE.local.md, 个人在该项目的私有覆盖 (通常 gitignore).
- **优先级:** 后加载者覆盖先加载者 (project 覆盖 user, 但 managed 不可被覆盖).
- **建议:**

- 每个文件 < 200 行, 太长可用 @path/to/file.md 引用拆分.
- 不要写”项目背景介绍”, 要写”agent 必须遵守的硬规则”.

Q8. Hooks 是什么? 列举几种你常用的 hook 场景.

参考答案:

- **Hook:** Claude Code/Codex 在生命周期事件 (例如 PreToolUse、PostToolUse、SessionStart、Stop) 上注册的**确定性回调**, 由 harness 而非 LLM 执行, 因此不会被模型遗忘或绕过.
- **类型 (Claude Code):** command (shell)、http (POST endpoint)、prompt (单轮 LLM 评估)、agent (多轮 LLM)、mcp_tool.
- **常见场景:**
 1. PostToolUse 命中 Edit|Write → 自动跑 prettier/eslint/gofmt.
 2. PreToolUse 命中 Edit(/.env*) → 阻止 (exit 2) 防止误改密钥.
 3. Stop → 发桌面通知告诉我 Claude 已经空闲.
 4. SessionStart → 输出当前 git 状态、环境变量, 让 agent 一上来就知道现状.
 5. PreToolUse 命中 Bash(rm -rf *) → ask 模式, 强制用户确认.

Q9. Permission Mode 有哪些? 各自适合什么场景?

参考答案:

模式	行为	适合场景
default	每个工具第一次使用时询问	不熟悉的新项目
acceptEdits	自动放行文件编辑和 fs 命令	已熟悉的项目, 想加速迭代
plan	只读模式, 禁止写操作	代码 review 或方案设计
auto	后台分类器判断是否安全	信任分类器的”半自动”模式
dontAsk	没在 allowlist 的一律拒绝	严格白名单生产场景
bypassPermissions	全部放行 (仍保留 rm -rf / 等熔断)	沙箱 / 容器 / dev container 内部

Q10. /compact 时, 哪些上下文会被保留, 哪些会丢失?

参考答案:

- 会保留 (重新注入):
 - System prompt
 - 项目根 CLAUDE.md + 未带 paths: 限定的 rules
 - Auto memory (从磁盘重读)
 - 对话摘要 (由 Claude 生成的 conversation summary)
 - 前 N 个高优先级 skill 描述 (25K token 预算内)
- 会丢失:
 - 历史的 tool call 详细输出 (例如读过的文件内容、bash 输出)
 - 路径范围限定的嵌套 CLAUDE.md / .claude/rules/*.md (等到下次匹配文件被读到时才会重新触发)
 - MCP tool 的完整 schema (只保留名字, 等下次调用时再 fetch)
- 实践建议: 不要把”必须长期记住的事情”放在对话里, 要么写进 CLAUDE.md, 要么用 auto memory 显式保存.

Q11. Plugin 和 Skill 的关系是什么?

参考答案:

- **Skill:** 单个能力单元 (一个 SKILL.md + 附属资源).
- **Plugin:** 一个打包发布的扩展包, 可以包含多个 skill, 同时还可以打包:
 - sub-agent 定义
 - hooks
 - MCP server
 - LSP server
 - 二进制可执行
 - 默认 settings
- 通过 .claude-plugin/plugin.json 描述元信息, 安装后 plugin 内的 skill 用命名空间调用: /<plugin-name>:<skill-name>, 避免冲突.

- 简言之: **skill** 是零件, **plugin** 是装好的整车.
-

Q12. 什么是 ToolSearch / Deferred Tools? 为什么要这样设计?

参考答案:

- **现象:** 启动时, MCP 工具和部分内置工具的完整 **JSON schema** 不会立刻加载, 只把名字告知模型. 当模型需要某个工具时, 通过 ToolSearch 把 schema 拉进来.
 - **原因:**
 1. **节省 token** —— 一个大型 MCP server 可能有几十上百个 tool, 全部 schema 加起来轻松上万 token.
 2. **降低注意力噪声** —— 不相关的工具长期挂在 prompt 里会干扰模型决策.
 3. **按需加载** —— 只有真正要用时才把详细签名带进来.
 - **代价:** 第一次使用某工具会多一次 ToolSearch round trip; 但比起 token 节省, 这笔交易很划算.
-

三、工作流篇

Q13. 你接到一个“中等复杂度的新功能”任务, 应该怎么用 **agentic** 工具拆?

参考答案 (一种典型流程):

1. 进入 **plan mode** (Shift+Tab 或对应快捷键), 只读探索代码.
2. 用 Explore sub-agent 并行搜索相关文件 (找入口、找数据流、找测试).
3. 用 Plan sub-agent 或主对话本身产出 **TDD 风格** 的方案:
 - 先写验收标准 / 测试用例
 - 再写最小实现
 - 再写验证策略
4. 用 ask-codex 或 ask-gemini 找第二意见 (cross-model review), 修补盲点.
5. 用 AskUserQuestion 让用户在关键设计岔路上拍板.
6. ExitPlanMode → 进入 acceptEdits 模式落地.
7. 写代码前先写 (或更新) 测试, 然后让 agent 实现, 实时跑测试.
8. 完成后用 /review 或 superpowers: requesting-code-review 自审.

9. 提交前过一遍 `verification-before-completion skill`.

Q14. 什么时候应该开 `sub-agent`, 什么时候不该?

参考答案:

- 应该开:
 - 要读 10+ 个文件做调研 (`Explore`).
 - 要跑大量 `grep / find /` 日志分析, 输出会很噪.
 - 要做独立的代码 `review`、安全 `review`.
 - 要并行做多个无依赖的子任务 (`dispatching-parallel-agents`).
 - 不该开:
 - 任务只需要 1-3 个工具调用 → 主对话直接做更快.
 - 任务需要主对话保留思考过程用于后续决策 → 开 `agent` 会丢上下文.
 - 任务需要多次交互式确认 → `sub-agent` 不能问用户.
 - 任务就是“读一个已知路径” → 直接 `Read`, 别用 `agent`.
-

Q15. 怎么避免主对话上下文被污染?

参考答案:

1. 大量读取 / 调研 → 用 `Explore sub-agent`, 只回传 `summary`.
 2. 长输出命令 → 用 `head/grep` 收敛, 或写进文件再让 `agent` 选读.
 3. 重复性任务封装成 `skill`, 避免把规则反复打字进对话.
 4. 及时 `/compact` 在阶段性里程碑之后压缩.
 5. 不要让 `agent` 反复读同一文件 —— 文件状态由 `harness` 维护, 改完不需要重读验证.
 6. 把“长期记住的事”写进 `CLAUDE.md / auto memory`, 不要靠对话里讲一次.
 7. `MCP` 工具按需加载 —— 不要一次性把所有 `MCP server` 都连上.
-

Q16. 代码出了 `bug`, 你会怎么用 `agentic` 工具调试?

参考答案:

1. 先用 `superpowers:systematic-debugging skill` 进入系统化排查模式.
 2. 不要急着改代码 —— 先复现, 写一个能稳定触发 bug 的最小用例 / 测试.
 3. 让测试先失败, 确认理解了 bug.
 4. 用 Explore 找出 bug 发生路径上的所有相关文件.
 5. 假设 → 验证 → 修复 (而不是猜测式打补丁).
 6. 修完后跑全部相关测试 + `verification-before-completion`.
 7. 复杂 bug 用 `codex:rescue` 让 Codex 做独立诊断, cross-check 思路.
-

Q17. 在多人协作的仓库里, 如何让 agentic workflow “团队化”?

参考答案:

- 共享层:
 - 仓库根 `CLAUDE.md` / `AGENTS.md` —— 团队所有人共享的硬规则 (代码风格、PR 流程、禁止事项).
 - `.claude/skills/`、`.claude/agents/`、`.claude/commands/` 全部入 git.
 - `.claude/settings.json` —— 共享 permission allowlist、hooks.
 - 个人层:
 - `CLAUDE.local.md` (gitignore) —— 个人偏好.
 - `.claude/settings.local.json` —— 个人 permission 覆盖.
 - 分发机制:
 - 把团队公共能力打包成 **plugin**, 放到内部 marketplace, 新成员一条命令 `/plugin install` 上手.
 - 审计:
 - hooks 在 `PreToolUse` 上对敏感操作记审计日志.
 - PR 模板要求贴出 agent 协作的关键决策点.
-

四、系统设计篇 (开放思辨)

以下题目无标准答案, 重点是看候选人的取舍框架和风险意识.

Q18. 设计一个“agentic 低代码平台”, 你会优先考虑哪些设计原则?

讨论方向:

1. 谁是 **source of truth**? Agent 生成的代码是 source, 还是低代码 DSL 是 source? 二者双向同步是个工程地狱, 必须二选一.
2. 可逆性与版本控制 —— agent 一键改 50 个组件后, 必须能 diff、能 revert、能 review.
3. 沙箱与权限分级 —— 平台用户的 agent 不能直接访问数据库, 必须经过权限网关 (类似 permission mode).
4. 上下文供给 —— 业务知识怎么进入 agent? 是要求用户写 CLAUDE.md 风格的指南, 还是从已有数据自动提炼?
5. 失败可观测 —— agent 调用第三方 API/MCP 的失败必须能回放, 而不仅是“它说失败了”.
6. 不要替代用户思考 —— 设计良好的 agentic 平台让用户减少打字, 但不减少决策; 关键岔路必须显式确认.
7. 可组合 —— 平台能力应该像 skill/plugin 一样可被组合, 而不是一堆魔法按钮.
8. 成本可见 —— 每个 agent 操作的 token / API / 时延都要前置可见, 否则用户会失控.

Q19. 设计一个企业内部的 “MCP 网关”, 你会如何处理权限、审计、限流?

讨论方向:

- **权限:** 谁能 list 哪些 tool? 一个 read-only 角色看不到 delete_* 工具的 schema, 减少误调用面.
- **审计:** 每次 tool call 落日志, 包含 user / agent_id / 参数 / 返回大小 / 耗时.
- **限流:** 同一 user 的 MCP tool 调用 QPS / 并发数, 防止 agent 跑飞.
- **数据脱敏:** Tool 返回值经过中间层脱敏 (PII、密钥), 再回到 agent.
- **降级策略:** 后端服务挂了, MCP 网关应返回 structured error 而不是超时, 让 agent 知道“这条路不通”.
- **版本管理:** Tool schema 改了怎么 backward compatible? 加版本号, 旧客户端继续看到旧 schema.

- **隔离:** 不同租户的 MCP server 进程隔离, 避免相互看到对方数据.
-

Q20. 如果让你设计 Claude Code / Codex 的多 agent 协作模型, 你会选择“星型 (主-子)”还是“网状 (互通)”? 为什么?

讨论方向:

- **星型优点:** 易调试、上下文可控、责任清晰; 缺点: 主对话成为瓶颈, 并行度有限.
 - **网状优点:** 接近真实团队, 并行高; 缺点: 易出现“agent 间循环对话”、“上下文爆炸”、“责任不清”、“调试地狱”.
 - **现实答案:** 大多数任务星型够用, 极少数复杂多角色任务才上网状, 而且要配合消息预算、总线程数上限、可观测的对话图等护栏.
 - **进一步思辨:** 多 agent 协作的 ROI 是否真的高于“一个更强的单 agent”? 如果模型能力继续提升, 多 agent 编排可能变成短期权宜之计.
-

Q21. 设计一个“AI 安全 review agent”, 要避免哪些坑?

讨论方向:

- **不能只看 diff** —— 安全问题往往出现在 diff 没改的“调用方”. 必须能扩展到上下文.
 - **不能盲信测试通过** —— 测试覆盖不到的 (反序列化、命令注入) 才是常出问题的地方.
 - **不能让 agent 自己跑攻击命令** —— review 阶段必须只读, 否则 review agent 反而成攻击面.
 - **要 cross-check** —— 用第二个模型 (例如 Codex) 独立 review 一遍, 降低单模型盲点.
 - **不要追求“零误报”** —— 安全 review 宁可误报也别漏报, 但要分级 (P0 / P1 / P2), 让人工聚焦.
 - **可解释** —— agent 必须能给出“为什么这是漏洞”的论证链, 而不是只丢一句“this is unsafe”.
-

Q22. 设计一个“长期记忆”机制 (类似 Claude Code 的 auto memory), 你会怎么决定记什么、不记什么?

讨论方向:

- **应该记:** 用户反复纠正过的偏好 (代码风格、命名)、项目级硬约束、易遗忘的 trivia (端口号、密钥位置).
 - **不应该记:** 一次性的临时对话、含有敏感数据的内容、过时的事实 (上次的 bug 早已修复).
 - **如何更新?** 写时合并 (避免重复)、定期 GC (太老的条目降权或删除)、用户可以一键审查 / 编辑 memory 文件.
 - **如何避免污染?** 把 memory 限定大小 (例如 25KB), 超出后按“使用频率 + 用户标记”取舍.
 - **跨项目共享 vs 项目独立?** 偏好类跨项目共享, 项目知识类不共享 —— 否则 A 项目的密钥被误用到 B 项目.
-

五、概念哲学篇 (开放思辨)

Q23. 为什么应该“保持 context 简单”? 一个塞满信息的 prompt 不是更好吗?

讨论方向:

- **注意力是有限资源** —— LLM 的注意力机制对相关内容的权重并非均匀, 过多无关信息会**稀释**关键指令的权重 (“lost in the middle”现象).
 - **token 成本** —— 每次推理都按全部 context 计费 / 计算; 长 context 直接拉高延迟和账单.
 - **可调试性** —— 出问题, 短 context 容易定位是哪条指令冲突; 长 context 几乎不可能复盘.
 - **可演化性** —— context 越简单, 越容易迭代 / 替换 / 重排; 复杂 context 像一团 spaghetti.
 - **哲学层面:** “简单”不是“少信息”, 而是“高信噪比”. 让 agent 知道它**需要知道的**, 而不是所有可能有关的.
-

Q24. Agent 应该“主动”还是“被动”? 它什么时候该问用户, 什么时候该自己决定?

讨论方向:

- **行动的可逆性** —— 可逆的 (本地改个文件) 自决; 不可逆的 (push、delete、send email) 必问.
 - **影响半径** —— 只影响本机 / 自决; 影响共享系统 (CI、PR、Slack) / 必问.
 - **不确定度** —— agent 自己心里没底的时候应该问, 而不是赌一把.
 - **用户当前可达性** —— 用户在线就多问; 后台 / 离线任务 (cron) 倾向于保守自决.
 - **设计哲学**: 真正好的 agent 不是“什么都不问”或“什么都问”, 而是**问对问题** —— 关键岔路问, 鸡毛蒜皮不打扰.
-

Q25. “Vibe Coding” 到底是 hype 还是范式革命?

讨论方向 (鼓励多元观点):

- **Hype 派论点**: 这只是更智能的自动补全; 复杂项目里 agent 仍然会犯低级错误; 维护成本被低估.
 - **范式派论点**: 编程的“主单位”从“行/函数”上升到“意图/约束”, 工程师角色从“打字员”变成“架构师 + 审稿人”; 软件交付速度的瓶颈正在被打破.
 - **中间立场**: 是范式革命, 但不是 1:1 替代 —— 它放大了高水平工程师的能力 (因为他们能给出好 spec 和好 review), 也放大了低水平工程师的破坏力 (因为他们也能快速产出垃圾).
 - **关键追问**:
 - 如果 agent 写代码, 知识在哪里沉淀? 是仓库 (CLAUDE.md) 还是个体大脑?
 - “代码 review” 的角色会不会比“写代码”更重要?
 - 教育、入门门槛、初级岗位会如何变化?
-

Q26. 一个 agent 反复改不对一个 bug, 你应该让它继续试, 还是关掉自己上?

讨论方向:

- **信号识别**: Agent 已经在“循环尝试同一类错误解” → 上下文已被污染, 继续多半无效.

- **沉没成本:** 已经烧了多少 token 不重要, 重要的是“再烧 X token 能否解决”, 概率 / 成本不划算就停.
 - **元问题:** Agent 卡住通常意味着**信息缺失**而非**推理失败** —— 此时应人工补信息 (贴日志、贴文档、贴失败用例), 而不是让它继续猜.
 - **健康习惯:** 给自己定一个上限 (例如“agent 改 3 次还不对就接手”), 避免无意识的 over-reliance.
 - **哲学:** Agent 是放大器, 不是替代品. 当它放大的是**死胡同**时, 越用越深.
-

Q27. 为什么“测试驱动开发 (TDD)”在 agentic 工作流里比传统工作流更重要?

讨论方向:

- **Agent 容易“自信地写错”** —— 它会编 API、编字段、编返回值, 测试是唯一的客观裁判.
 - **测试是和 agent 沟通的契约** —— 你不需要详细描述“怎么做”, 你只需要说“做完后这个测试必须过”.
 - **TDD 给 agent 提供了反馈循环**** —— 没有失败的测试, agent 不知道自己错没错.
 - **TDD 强制把模糊需求变成可执行规约** —— 这恰好是 agent 最需要的输入.
 - **风险:** 反向地, 如果 agent 自己写测试又自己写实现, 它可能“为了过测试而过测试”. 关键测试 (验收级、安全级) 应由人写或独立 review.
-

Q28. “agent 写的代码可读性差”该怎么办? 这是工具问题还是用法问题?

讨论方向:

- **工具侧:** 模型确实有“过度抽象 / 过度防御 / 过度注释”的倾向; 系统 prompt 里限定 (例如 Claude Code 默认 prompt 就有“不要加无意义注释”) 能改善.
- **用法侧:** 用户没给约束 / 没 review / 接受 first try → 垃圾代码堆积. 这是**用法问题**, 不是工具问题.
- **根本解:**
 1. 在 CLAUDE.md 里写明可读性硬规则 (函数长度、命名、注释规约).
 2. 把“简化 / 重构 / 删冗余”作为独立 step, 而不是寄希望于 first try.
 3. 把 review 标准外置成 skill (如 simplify、code-reviewer), 强制 agent 自审.

- **更深一层:** agent 写的代码“读不懂”, 还是因为读的人没有花时间理解. 如果完全不读, 那相当于把生产代码外包给了一个不会被追责的实习生.
-

Q29. “我让 agent 跑了一整晚, 它做了什么我看不懂” —— 这是 agent 该解决的问题还是用户的问题?

讨论方向:

- **可观测性是工具方的责任** —— agent 必须能输出可审计的操作序列 (file diff、command log、决策点).
 - **理解力是用户方的责任** —— 工具再好, 跨夜 1000 次 tool call 也不可能逐条审; 必须**事前约定边界** (例如 permission mode = plan, 或者只让它跑特定子任务).
 - **设计哲学:** 越长的自主任务, 越需要**强约束 + 阶段 checkpoint**, 而不是“放手让它跑然后审”.
 - **极端情形:** 完全无人监督的长跑 agent, 风险类似于“无人监督的初级实习生 + sudo”, 在生产环境基本不能接受.
-

Q30. 终极思考题: agentic 工具最终会让“程序员”这个职业消失吗?

讨论方向 (没有正确答案):

- **不会消失, 会重塑:** 写代码的部分被外包, 但**理解需求 / 拆分系统 / 判断对错 / 承担责任**这部分永远需要人.
- **会消失一种“程序员”:** 那种把工作完全定义为“打字实现 spec”的中下层岗位, 风险显著.
- **新角色会出现:**
 - “Agent 架构师” —— 设计 agent 协作图、skill 库、context 策略.
 - “AI 审稿员” —— 能高速 review agent 输出的资深工程师.
 - “Context 工程师” —— 把领域知识结构化为 agent 可消化的形式.
- **历史类比:** 编译器没有让汇编程序员消失, 但确实让“手写 x86”的人变少了 —— 程序员的抽象层次会再上一层, 而不是被取代.
- **个人立场题:** 你认为 5 年后, 你日常工作的什么比例还是“自己打字”? 30%? 5%?

附录: 推荐学习路径

1. **入门:** 读 <https://code.claude.com/docs/llms.txt>, 跑通 /skill, /agent, /plugin, /compact, /memory.
 2. **进阶:** 把自己常做的 3 件事 (commit message、PR 摘要、跑测试) 各写成一个 skill.
 3. **高级:** 给团队写一个 plugin, 内含 skills + agents + hooks + 一个 MCP server.
 4. **专家:** 研究 hooks 在 PreToolUse 上的复杂决策、Agent Team 实验特性、长任务 (overnight) 的 checkpoint 设计.
-